Traffic Detection using YOLOv5

Krzysztof Nazar 184698
Hubert Nacmer 184546

1. Task Description

The dataset comes from Kaggle: Traffic Detection Project. The objective of this project was to develop and optimise a traffic detection model using the YOLOv5 framework. The dataset consists of annotated traffic camera images collected from multiple countries, capturing a diverse range of road conditions, lighting scenarios, and traffic densities. Each image is labelled with bounding boxes for five distinct object categories: bicycle, bus, car, motorbike, and person. These annotations enable precise object detection and localisation, crucial for real-world applications in traffic monitoring and road safety.

2. Dataset Exploration

The dataset comprises images of a fixed size: 640×640 pixels, in RGB colour format. This uniform resolution simplifies preprocessing and model input handling. The dataset is already divided into three distinct subsets: training, validation, and test, each containing a different number of images. The training, validation, and test splits are distributed such that 88% of the dataset is used for training, while 8% is used for validation, and the final 4% is reserved for testing. Figure 1 illustrates the distribution of images across these splits.

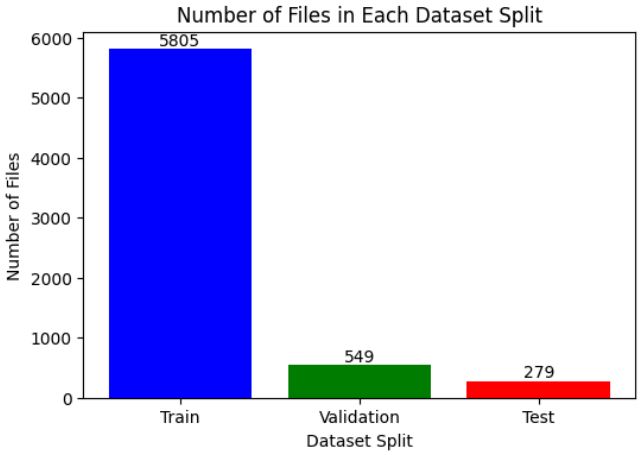

Figure 1. Number of Files in Each Dataset Split - Train, Validation, and Test.

To gain insights into the dataset structure, we examined the bounding box annotations. Figure 2 presents the distribution of bounding boxes across the training, validation, and test sets. The distributions are unimodal, exhibiting a long right tail, which indicates that while most images contain between 3 to 15 annotated objects, some images have significantly more, sometimes reaching even 40 annotations per image.

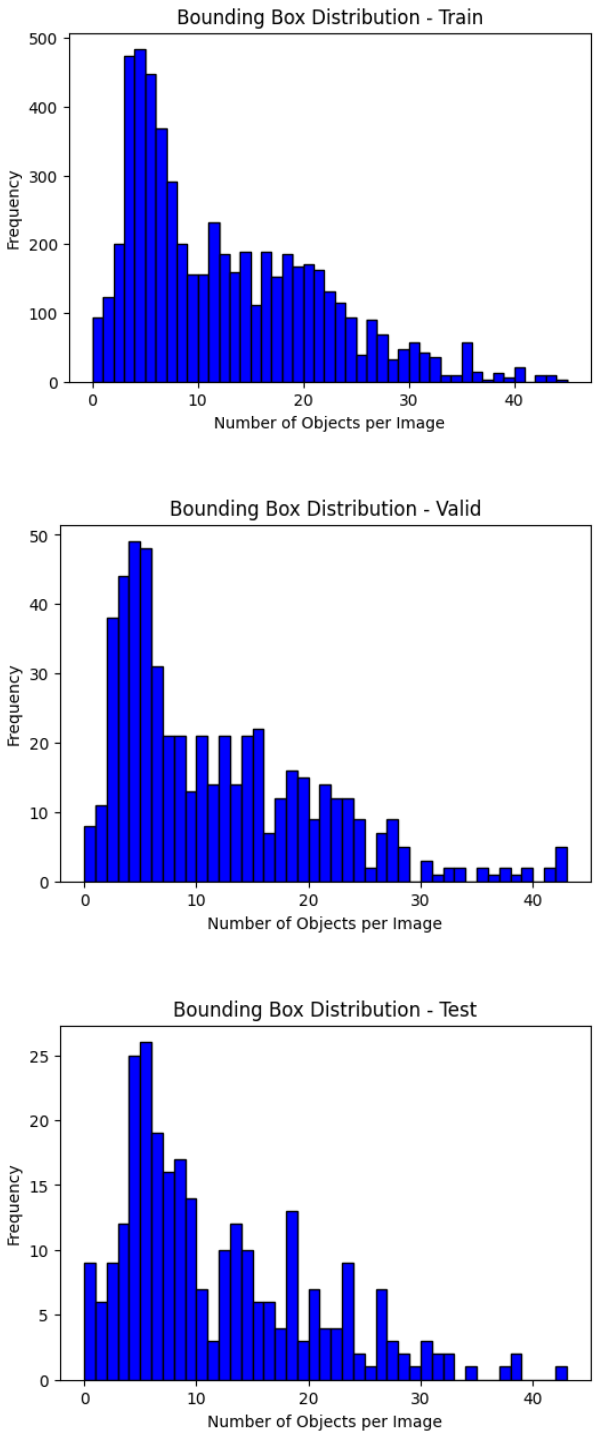

Figure 2. Bounding Box Distribution in each Dataset Split - Train, Validation, and Test.

Next, we wanted to understand the class distribution in each split, while it is essential to ensure that all object categories are well-represented in the dataset. This distribution is visualised in Figure 3. The plots reveal an imbalance, as certain classes such as car, motorbike, and person appear significantly more frequently than others - bicycle and bus. However, after examining sample images, it appears that this distribution reflects real-world traffic scenarios and we will not change it.

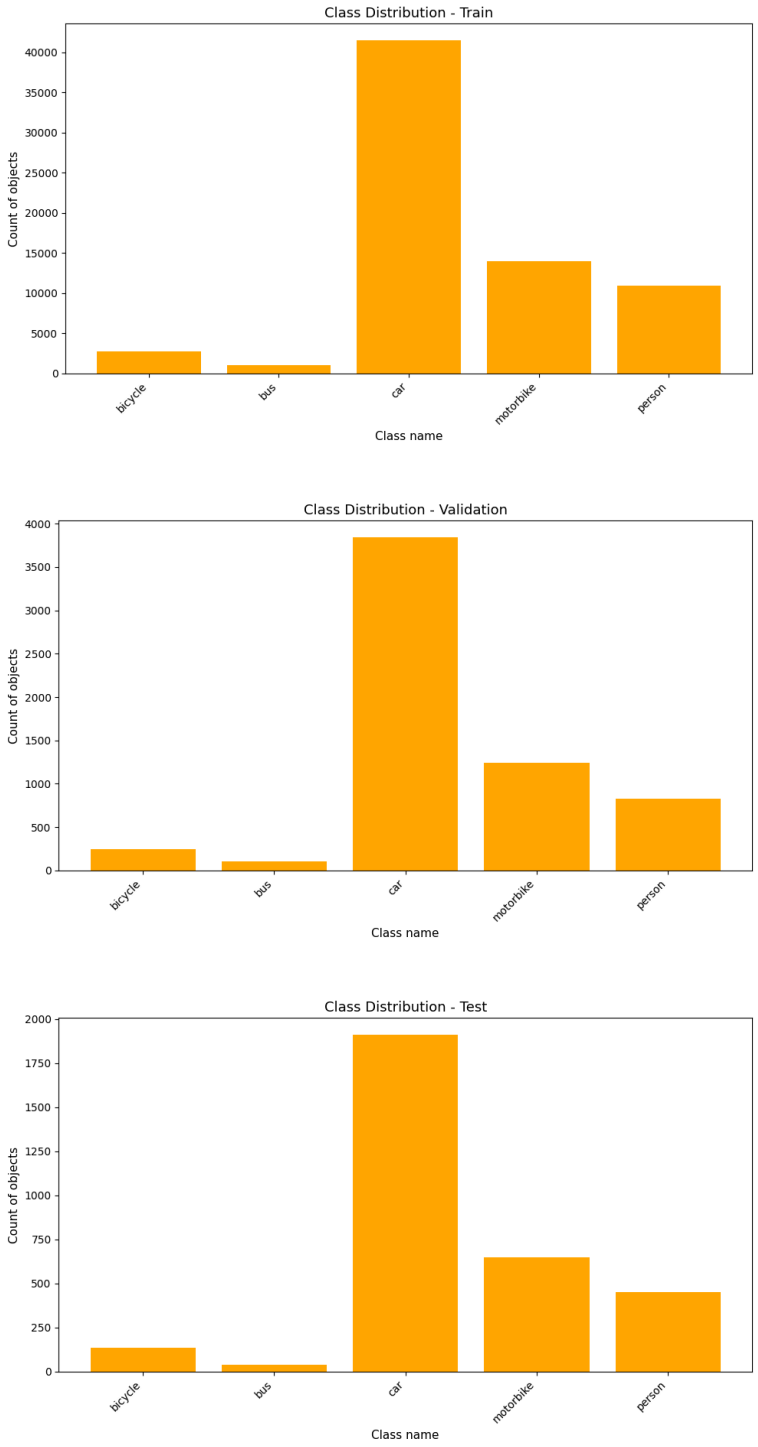

Figure 3. Class distribution in each split in the dataset.

A qualitative analysis of the dataset's annotation quality was performed by visualizing randomly selected images from each split, as depicted in Figure 4. While the annotations demonstrated a high degree of accuracy, instances of unannotated objects were observed, indicating potential inconsistencies in the labeling process. In some images there are some objects that were not annotated even though they are present in the picture.

Figure 4. Sample images from each dataset split with their annotations.

3. Data Augmentation

To enhance the training dataset, additional samples were generated using a set of transformations tailored to real-world traffic detection scenarios. These transformations were carefully selected to improve model robustness by simulating various environmental and camera conditions. For each original image, three augmented copies were created, effectively increasing the size of the training set by a factor of four.

There were 5 augmentations techniques applied using the albumentations library[1], which offers a wide range of transformations for image data in computer vision problems. Each technique has some predefined probability of being applied to an image during the augmentation process:

[1] https://pypi.org/project/albumentations/

- **Horizontal Flip (p=0.5):** Simulates mirrored traffic scenes, improving robustness to camera placement variations.
- **Shift, Scale, Rotate (p=0.6):** Mimics camera movement and perspective changes, enhancing object recognition under distortions.
- **RGB Shift (p=0.3):** Simulates diverse lighting conditions, making the model less sensitive to color fluctuations.
- **Brightness/Contrast (p=0.3):** Adapts the model to varying weather and lighting, simulating real-world exposure changes.
- **Blur (p=0.1):** Replicates environmental blur (rain, fog, motion), improving object detection in poor visibility.

A randomly selected image from the training split, along with its three augmented versions, is presented in Figure 5. These images visually demonstrate the impact of the applied transformations on the dataset. The augmented versions showcase how the model is exposed to different perspectives, lighting conditions, and slight distortions, which help improve its generalisation ability. By introducing these variations, the model learns to identify objects in diverse real-world scenarios, making it more robust to changes in camera angles, environmental factors, and motion blur.

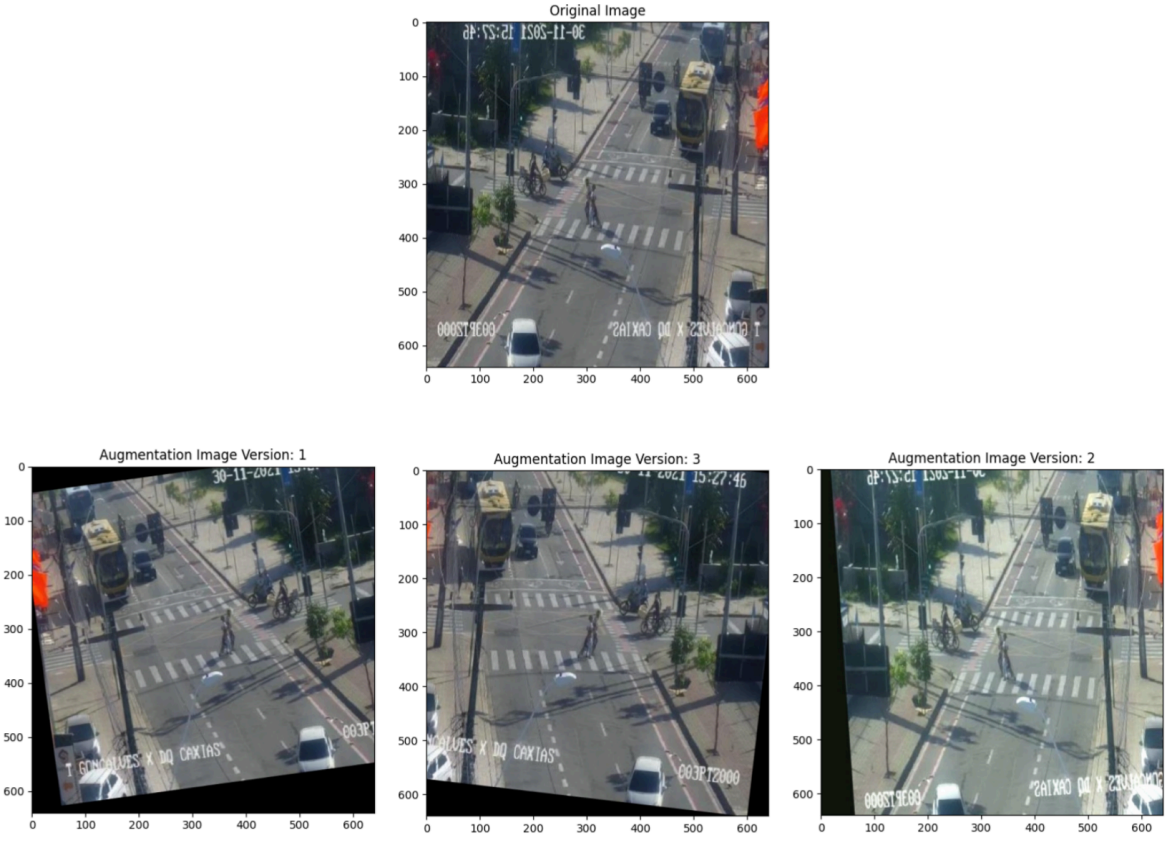

Figure 5. Image from the train split and its three augmented versions.

4. Choosing Baseline Model and Evaluation Metrics

Training Baseline Model

To establish a performance benchmark for our traffic detection project, we began by training a baseline model using the YOLOv5s architecture. For training we used NVIDIA GeForce RTX 4070 Ti, with 12GB of VRAM[2]. The YOLOv5s model[3] was selected for several key reasons:

- **Efficiency and Speed:** YOLOv5s is known for its relatively small size and ease of training, making it suitable for initial experimentation and rapid prototyping.
- **Dataset Compatibility:** Our dataset's annotations were specifically formatted for the YOLOv5 framework, simplifying the training process.
- **Established Performance:** YOLOv5s has demonstrated strong performance in various object detection tasks, providing a solid foundation for our project.

The model is a network consisting of 214 layers and approximately 7.03 million parameters.

The most important metrics that were monitored during training, validation, and evaluation of the model were:

- **Mean Average Precision (mAP):** mAP measures the average precision across different recall values. It provides a comprehensive evaluation of the model's accuracy in detecting objects, considering both precision and recall. A higher mAP indicates better overall performance.
- **Precision:** Precision quantifies the proportion of detected objects that are actually correct. It answers the question: "Of all the objects the model detected, how many were accurate?" High precision means fewer false positives.
- **Recall:** Recall measures the proportion of actual objects in the image that the model successfully detected. It answers the question: "Of all the actual objects in the image, how many did the model find?" High recall means fewer false negatives.

The three losses described below were also monitored:

- **Box Loss:** The box loss quantifies the error in the predicted bounding box coordinates.
- **Object Loss:** The object loss measures the confidence of the model in detecting an object within a bounding box.
- **Class Loss:** The class loss measures the error in the predicted class labels.

While the box, object, and class losses were monitored, our primary focus was on mAP, precision, and recall, as we believe that in our case these metrics provide a more direct assessment of the model's object detection performance.

---

[2] https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4070-family/
[3] https://github.com/ultralytics/yolov5

5. Baseline Model - training using train split and short evaluation using validation split

The baseline model was trained exclusively on the original, unaugmented training split. The training setup was as follows:

- **Model:** YOLOv5s
- **Epochs:** 50
- **Batch Size:** 60
- **Image Size:** 640x640 pixels

The training process was monitored closely, and the following plots shown in Figure 6 summarize the behaviour of the analyzed metrics over the 50 epochs. These plots provide insights into the model's learning progress and convergence.

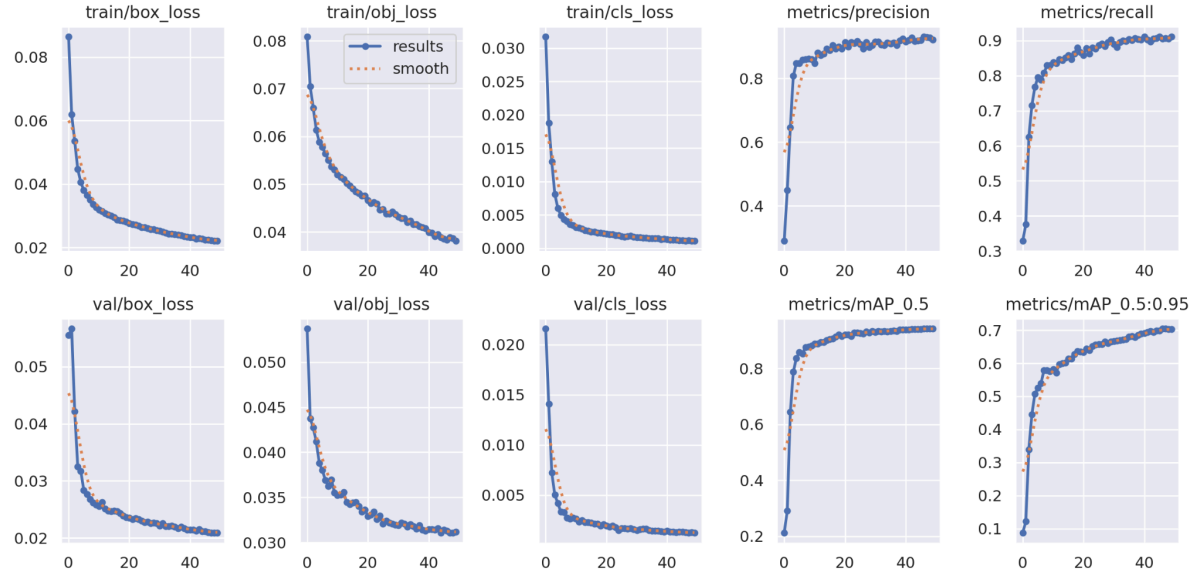

Figure 6. Training and validation metrics values during training the baseline model using the default training split.

All of the presented loss plots (box loss, object loss, class loss) show a clear decreasing trend over the 50 epochs, indicating that the model is learning and converging. The decrease is more rapid in the initial epochs and gradually levels off, suggesting that the model is stabilizing. The validation loss plots (val/box_loss, val/obj_loss, val/cls_loss) closely follow the training loss plots, suggesting that the model is generalizing well and not overfitting to the training data. Other metrics such as precision, recall, and mAP show a consistent increasing trend, reflecting the model's improving performance over time.

Model summary on training data on the validation split is shown in Table 1. Overall, the baseline model performs well on the validation split, achieving high precision (0.928), recall (0.908), and mAP50 (0.942) across all classes. The model excels in detecting cars, buses, and bicycles, demonstrating strong performance in these categories. However, it shows slightly lower performance in detecting motorbikes and persons, particularly in mAP50-95, indicating potential challenges in precise localization for these classes.

Table 1. Baseline model summary on the validation split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	549	6270	0.928	0.908	0.942	0.705
bicycle	549	250	0.936	0.937	0.947	0.751
bus	549	108	0.925	0.944	0.975	0.861
car	549	3842	0.94	0.945	0.966	0.769
motorbike	549	1238	0.942	0.855	0.921	0.576
person	549	832	0.899	0.858	0.902	0.567

Finally, Figure 7 displays the confusion matrix, which shows how well the model's predictions align with the actual object classes in the validation data. This matrix is a standard output of YOLOv5's *train.py* script during validating the training process.

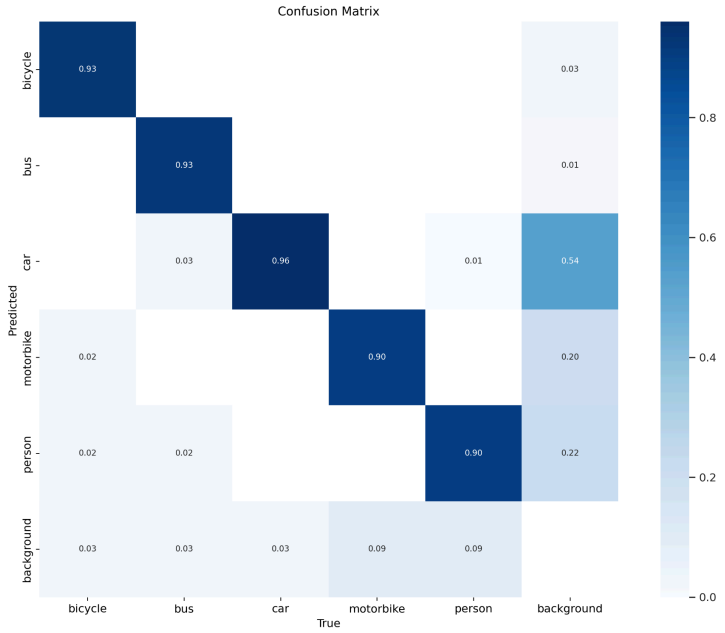

Figure 7. Confusion matrix of baseline model using validation split.

The diagonal elements for bicycle (0.93), bus (0.93), car (0.96), motorbike (0.90), and person (0.90) are high. This indicates that the model is generally accurate in detecting these classes. This is a very good indicator and it shows that the model is performing well.

However, there is high value in the car-background cell indicating that the model often predicts a car while it should be predicted as a “background” region. This indicates that the model has some difficulty correctly identifying when no objects are present. It means that the model is predicting a car in a region of the image that the annotators did not label as a car, resulting in more false positive predictions. This issue might be caused by the quality of data. As shown in Figure 8, in some images the annotations are missing. Hence, the model seems to be learning correctly and predicts the regions in images where the cars are actually presented, but they were not labelled.

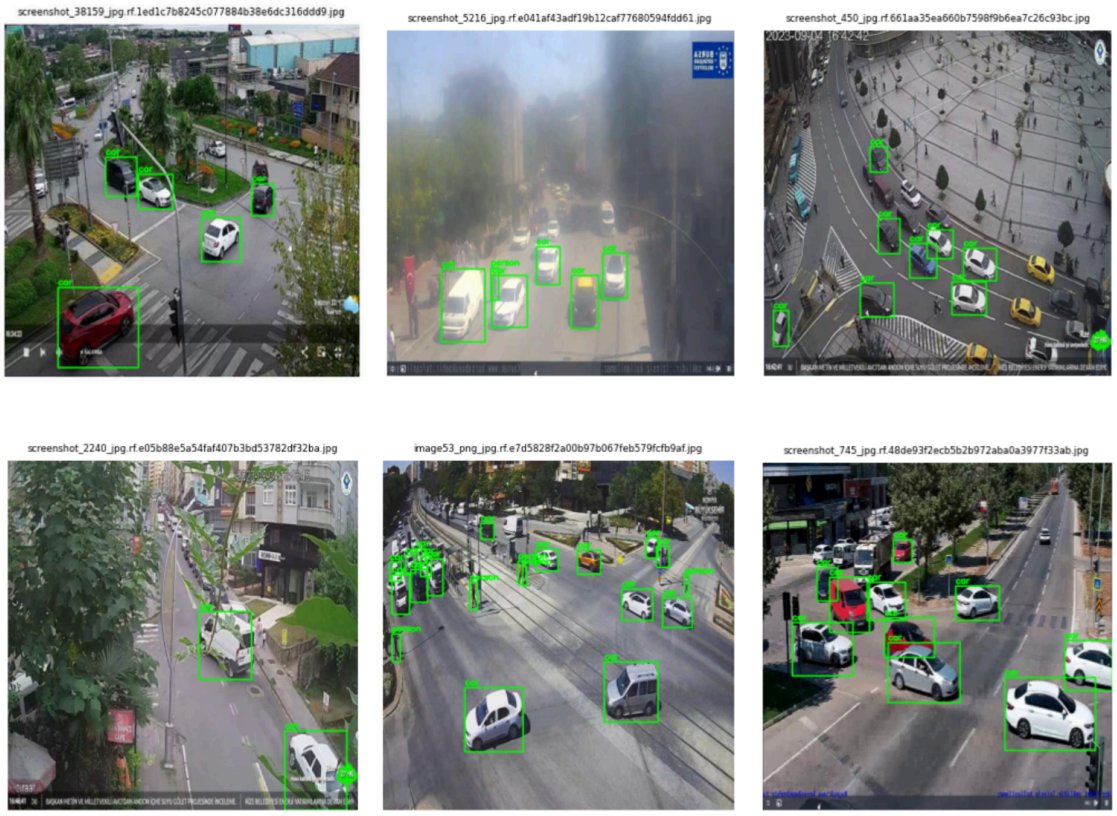

Figure 8. Examples of images where objects were not cars were not annotated fully correctly.

6. Baseline model evaluation using test split

Table 2 summarises the evaluation of the baseline model. Based on these observations, we can say that the model demonstrates excellent performance in detecting bicycles, buses, and cars, achieving high precision and recall values (above 0.92 for all three classes). This indicates that the model is highly reliable in identifying these object categories.

On the other hand, the model's performance is slightly lower for motorbikes and persons, with recall values of 0.851 and 0.84, respectively, and lower mAP50-95 scores (0.599 and 0.593). This suggests that the model may have more difficulty in accurately detecting and localizing these two classes.

Table 2. Baseline model summary on the test split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	279	3188	0.941	0.903	0.948	0.721
bicycle	279	134	0.973	0.948	0.981	0.795
bus	279	41	0.927	0.931	0.965	0.848
car	279	1911	0.945	0.946	0.966	0.771
motorbike	279	650	0.936	0.851	0.92	0.599
person	279	452	0.922	0.84	0.906	0.593

The confusion matrix showing the model's predictions and the ground truth in the test split is shown in Figure 9. The results support the numerical summarisation with metrics, showing high accuracy (dark blue diagonals) for all classes. Additionally, the model's difficulty with the background class, which was seen also when analysing the predictions for the validation split, indicates a need for further refinement to reduce false positives.

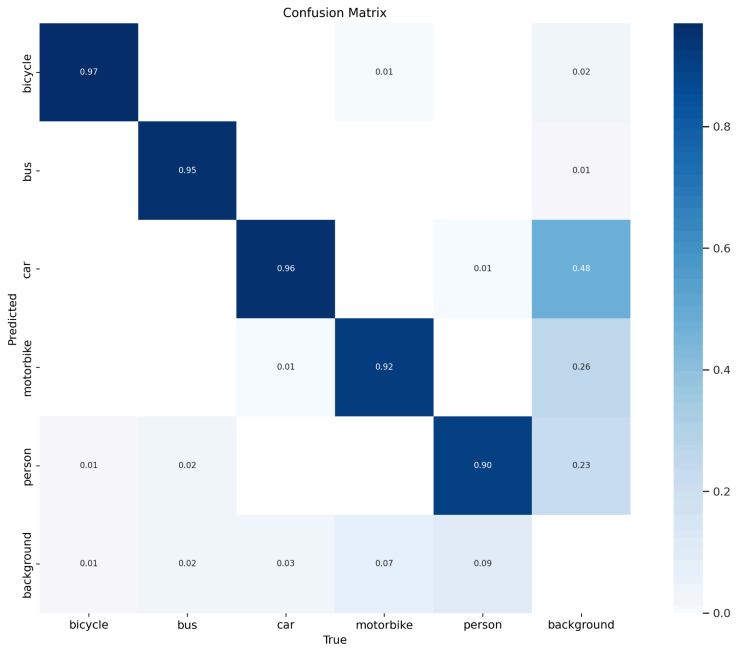

Figure 9. Confusion Matrix of baseline model using test split.

7. Hyperparameter optimisation in the baseline model

To further enhance the performance of our YOLOv5s model the *evolve* method was employed. It is a built-in hyperparameter optimization tool within the YOLOv5 framework[4].

The *evolve* function in YOLOv5 operates by creating a population of models, each with slightly different hyperparameter values. These models are then trained for a specified number of epochs, and their performance is evaluated using a fitness function - in our case the mAP50 value. The top-performing models are selected as "parents" for the next generation, and their hyperparameters are combined and mutated to create new offspring models. This process is repeated for a set number of generations, gradually converging towards an optimal hyperparameter configuration.

This method was chosen, because it allows for an automated and efficient way to tune the hyperparameters of the model. The genetic algorithm provides a good balance between exploration (trying new hyperparameter combinations) and exploitation (refining promising combinations). The evolve method is specifically designed for YOLOv5, making it well-suited for optimizing the model's performance on object detection tasks.

[4] https://docs.ultralytics.com/yolov5/tutorials/hyperparameter_evolution/

The parameters used during the hyperparameters tuning were very similar to the training parameters, but this time the number of generations was added:

- **Model:** YOLOv5s
- **Epochs:** 50
- **Batch Size:** 60
- **Image Size:** 640x640 pixels
- **Number of Generations:** 20

The evolve method ran for 8 iterations as it is designed to stop, when the model stops improving. The best-performing hyperparameters were saved in the *hyp_evolved.yaml* file and they can be easily used during next training sessions. The best generation was the 5th generation. The results using the hyperparameters from this generation are as follows:

- **metrics/precision:** 0.92468
- **metrics/recall:** 0.90458
- **metrics/mAP_0.5:** 0.94256
- **metrics/mAP_0.5:0.95:** 0.70695
- **val/box_loss:** 0.026674
- **val/obj_loss:** 0.039998
- **val/cls_loss:** 0.0016186

The plots in Figure 10 illustrate the results of the hyperparameter tuning process using the evolve method in YOLOv5. Each subplot represents a different hyperparameter, showing its relationship with the mAP_0.5 fitness metric. The red "x" marker indicates the hyperparameter value that resulted in the best mAP_0.5 score (0.94256), which is also included in the title of each subplot.

The plots show the exploration of different hyperparameter combinations across generations. The concentration of yellow points around the best hyperparameter values suggests that the evolve method effectively converged towards an optimal configuration. These plots provide a visual representation of the hyperparameter optimization process, highlighting the best hyperparameter values and revealing insights into the model's sensitivity to different settings.

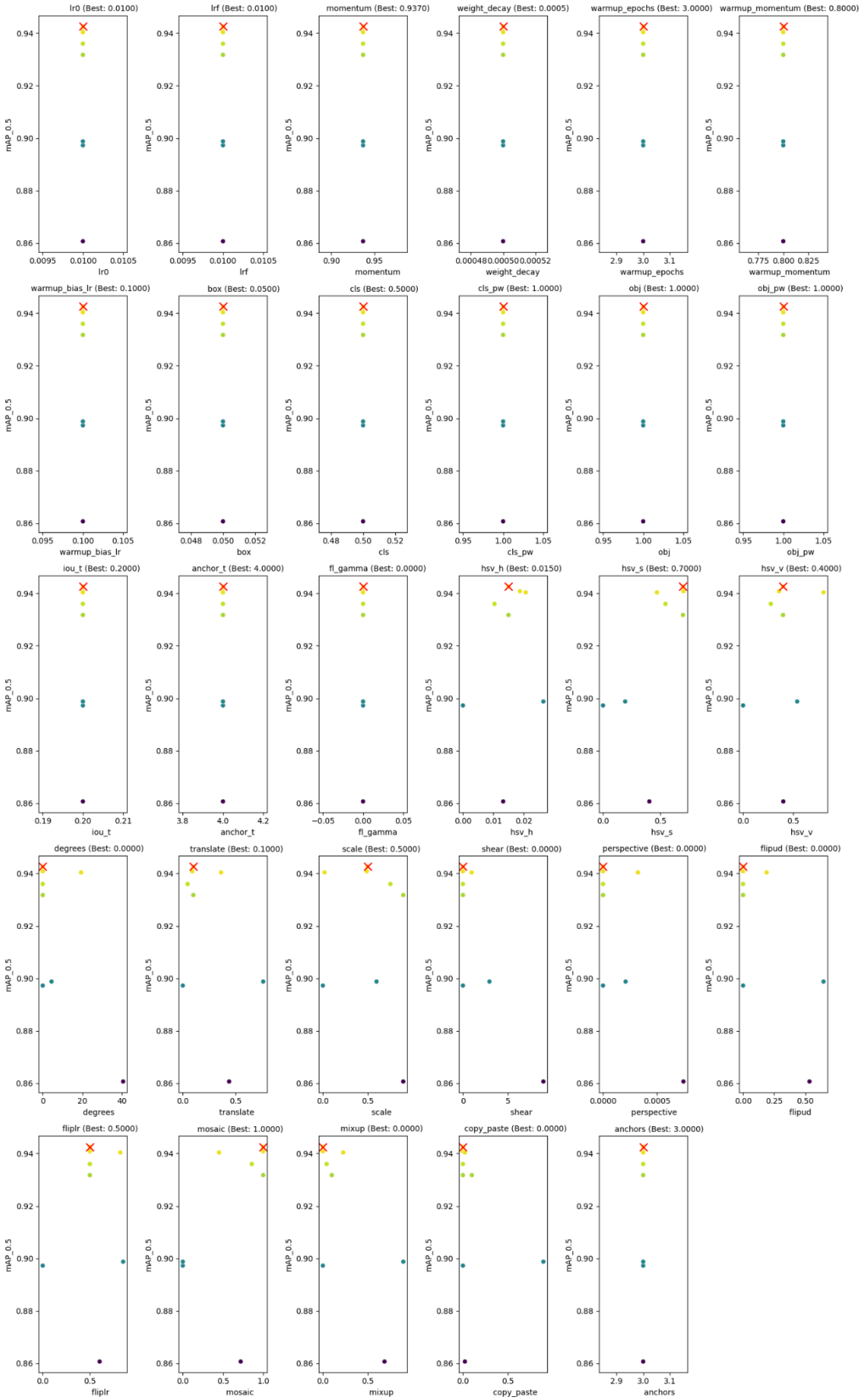

Figure 10. Hyperparameter tuning of the baseline model using the evolve method.

8. Model with tuned hyperparameters - training using train split and short evaluation using validation split

Following the hyperparameter optimization phase, we proceeded to train the YOLOv5s model using an expanded dataset that included both the original training images and their augmented versions. Each image from the training split was used to generate three additional synthetic images, effectively quadrupling the size of the training dataset. This augmentation strategy aimed to enhance the model's robustness and generalization capabilities by exposing it to a wider range of variations in traffic scenes, lighting conditions, and camera perspectives.

To accommodate the augmented data, the *data.yaml* file was modified to include both the original and augmented image directories for the training split.

Initially, we maintained the training duration at 50 epochs, consistent with the baseline model training and hyperparameter optimization phases. However, upon analyzing the training metrics' plots, we observed that the model's learning curve had not fully plateaued within 50 epochs. Given the increased dataset size, we extended the training duration to 60 epochs to ensure adequate convergence and to fully leverage the benefits of the augmented data. This adjustment allowed the model sufficient time to learn the expanded distribution of features and patterns introduced by the augmentation techniques.

Plots in Figure 11 present training and validation loss curves (box, object, class) and performance metrics (precision, recall, mAP/0.5, mAP/0.5:0.95) over 60 epochs of training using the extended training data.

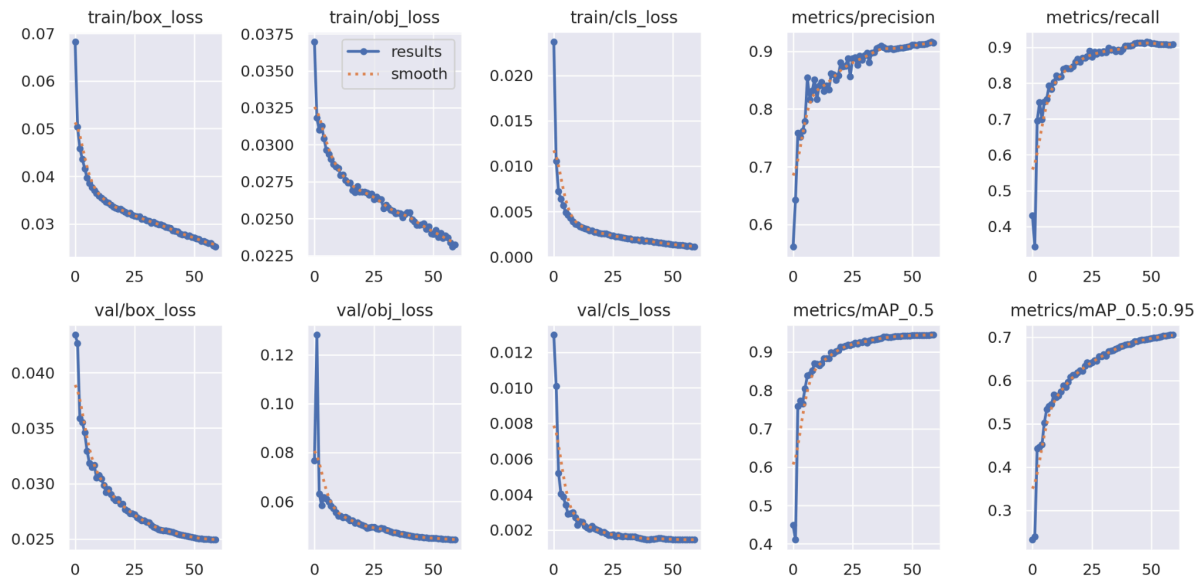

Figure 11. Training process of model with tuned hyperparameters using extended training data.

All loss curves show a consistent decreasing trend, indicating that the model is learning effectively. The curves flatten out towards the end, showing that the model has converged. There are no signs of overfitting, as the validation losses track closely with the training losses.

Precision, recall, mAP/0.5, and mAP/0.5:0.95 all show a consistent increasing trend, confirming that the model's performance improves with training. These metrics stabilize towards the end of training, aligning with the convergence of the loss curves.

When looking at the model summary presented in Table 3, it is clear that the model performs very well, with a precision of 0.911, recall of 0.903, mAP50 of 0.94, and mAP50-95 of 0.697 across all classes. The mAP_0.5:0.95 scores are significantly lower than the mAP_0.5 scores, which means that the model has trouble with accurate localization of the detected objects.

Table 3. Model with tuned hyperparameters summary on the validation split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	549	6270	0.911	0.903	0.94	0.697
bicycle	549	250	0.943	0.928	0.94	0.75
bus	549	108	0.926	0.931	0.972	0.849
car	549	3842	0.904	0.948	0.969	0.757
motorbike	549	1238	0.912	0.864	0.922	0.575
person	549	832	0.87	0.844	0.898	0.556

Confusion matrix shown in Figure 12 indicates that the model shows very good performance in detecting objects of all classes in the validation split. As seen in the earlier cases when evaluating the model, the background class has some misclassifications, especially in case of the car class, indicating that the model has trouble distinguishing the background from the objects and it often predicted that a car is present in the given region in a picture, even though these objects are not labelled in these images.

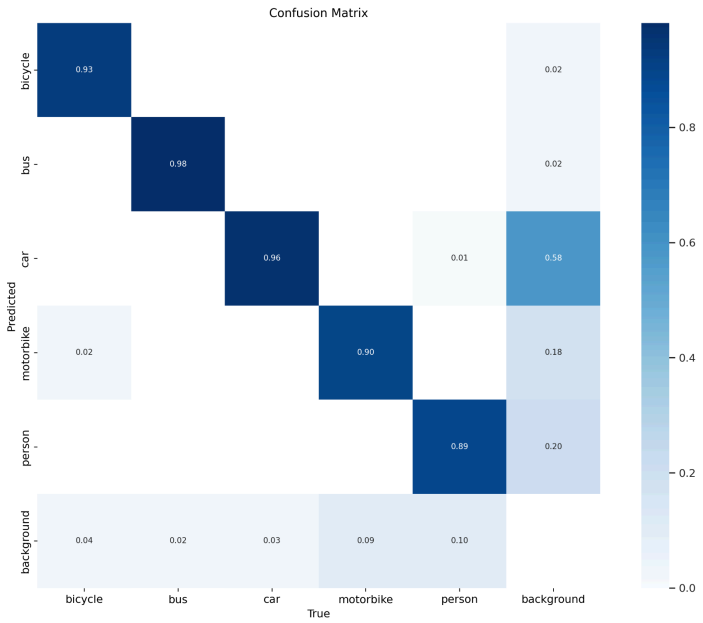

Figure 12. Confusion Matrix of model with tuned hyperparameters using validation split.

9. Model with tuned hyperparameters and augmented data - evaluation using test split

The evaluation of the YOLOv5s model, trained with tuned hyperparameters and augmented data, on the test split demonstrates generally strong performance. The overall precision is 0.937, recall is 0.901, mAP50 is 0.943, and mAP50-95 is 0.71, indicating a well-performing model. The model has an excellent performance in detecting bicycles, buses, and cars, with high mAP50 values (above 0.95 for these classes). The model's performance is comparatively lower for motorbikes and persons, particularly in terms of mAP50-95 (0.594 and 0.58, respectively). This suggests challenges in precise localization for objects of these two classes.

Table 4. Model with tuned hyperparameters summary on the test split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	279	3188	0.937	0.901	0.943	0.71
bicycle	279	134	0.976	0.94	0.977	0.781
bus	279	41	0.974	0.9	0.956	0.84
car	279	1911	0.913	0.94	0.964	0.757
motorbike	279	650	0.925	0.87	0.922	0.594
person	279	452	0.899	0.856	0.899	0.58

The confusion matrix supports the numerical findings, showing very high accuracy (dark blue diagonals) for all classes. As seen before, a notable misclassification persists between car and background, where a significant portion of background regions are predicted as car (0.58).

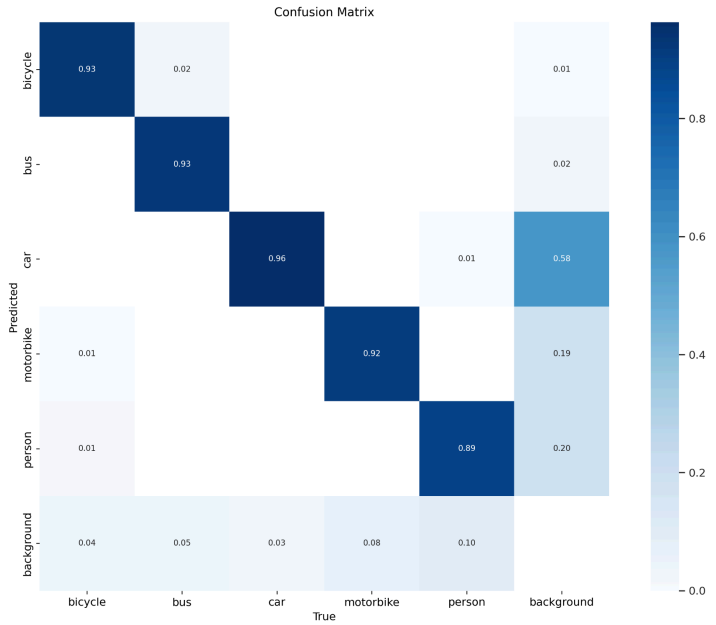

Figure 13. Confusion Matrix of model with tuned hyperparameters using test split.

10. Final Improvements

The final improvement that was added was Flip mosaic method found in research paper titled “Real-Time Vehicle Detection Based on Improved YOLO v5”[5]. This augmentation method uses mosaic composition of images, which is then flipped in multiple ways and combined again in a mosaic of flipped mosaics. It is done in order to improve detection of occluded objects in images. Method is presented in figure 14.

Figure 14. Flip mosaic method[5].

If any errors are encountered during mosaic composition, the composition is discarded for problematic images.

Chosen augmentation resulted in slight improvement in general precision but degraded in general recall and mAP.

Table 5. Model augmented with mosaic summary on the test split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	279	3188	0.933	0.904	0.945	0.707
bicycle	279	134	0.973	0.933	0.978	0.761
bus	279	41	0.974	0.914	0.957	0.838
car	279	1911	0.908	0.944	0.966	0.752
motorbike	279	650	0.928	0.868	0.925	0.599
person	279	452	0.882	0.861	0.899	0.586

[5] Zhang Y, Guo Z, Wu J, Tian Y, Tang H, Guo X. Real-Time Vehicle Detection Based on Improved YOLO v5. *Sustainability*. 2022; 14(19):12274. https://doi.org/10.3390/su141912274 (https://www.mdpi.com/2071-1050/14/19/12274)

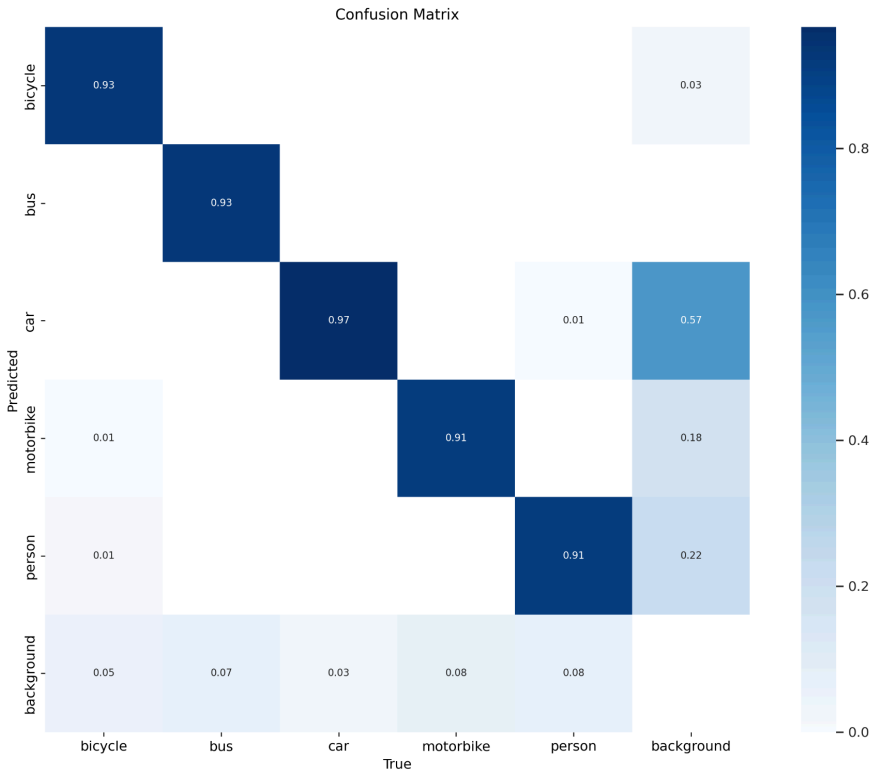

Figure 15. Confusion Matrix of model with mosaic augmentation using test split.

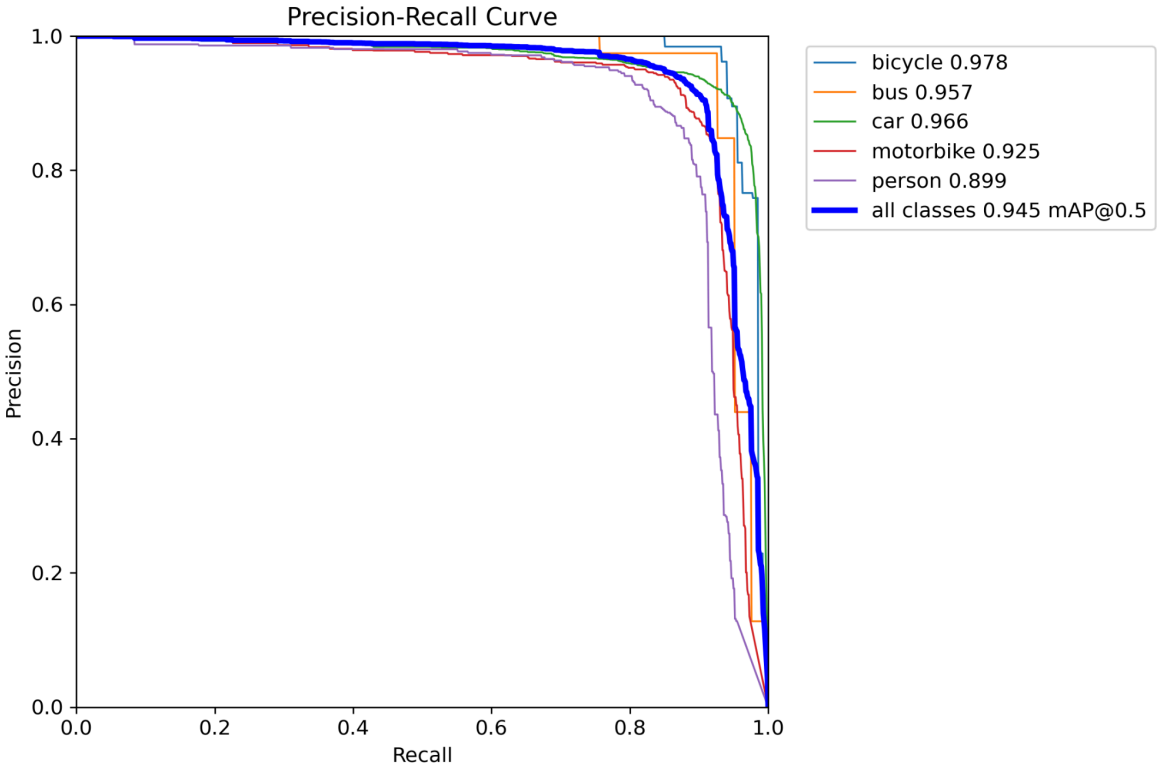

Figure 16. PR Curve of model with mosaic augmentation using test split.

11. Results Comparison

In this section we will compare one of the solutions acquired from kaggle discussion tabs[6].

Table 6. Our solution results on the test split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	279	3188	0.933	0.904	0.945	0.707
bicycle	279	134	0.973	0.933	0.978	0.761
bus	279	41	0.974	0.914	0.957	0.838
car	279	1911	0.908	0.944	0.966	0.752
motorbike	279	650	0.928	0.868	0.925	0.599
person	279	452	0.882	0.861	0.899	0.586

Table 7. Kaggle solution results on their validation split.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	549	6270	0.778	0.742	0.806	0.545
bicycle	189	250	0.911	0.653	0.787	0.558
bus	81	108	0.658	0.873	0.892	0.726
car	520	3842	0.848	0.896	0.935	0.68
motorbike	331	1238	0.734	0.748	0.765	0.443
person	196	832	0.741	0.541	0.651	0.319

The kaggle solution table is computed on the validation set created by the solution authors and ours is based on our test set. In this example our solution has better results in every category.

12. Conclusion

This project successfully trained a YOLOv5-based traffic detection model and optimised its performance using hyperparameter tuning and data augmentation. While our results approach state-of-the-art benchmarks and are better than some of sample kaggle solutions,

---

6 https://www.kaggle.com/code/mahedihasan1234/object-detection-using-yolov8/notebook#Model-Validation-as-a-separate-step

additional modifications such as integrating attention-based architectures or ensemble methods could further enhance accuracy. Future work includes exploring transformer-based object detection models for real-time traffic analysis.